Institución Universitaria

Facultad de Ciencias Exactas y Aplicadas
Física Computacional 1
Robinson Longas Bedoya
robinsonlongas3586@correo.itm.edu.co
robinsonlongas@itm.edu.co

► **Herramientas de Python para el cálculo:derivación e integración**

Bibliografía

- *Computational Physics*, Mark Newman.
- *Programming for computations - Python*, Evein Linge and Petter Langtangen.
- *Computational Physics with Python*, Eric Ayars.
- *Computational mathematics: An introduction to Numerical Analysis and Scientific Computing with Python*, Dimitrios Mitsotakis.

Derivación

Recordemos, desde la cinemática básica, si un cuerpo recorre una distancia de $\Delta x = 100$km en un tiempo de $\Delta t = 2$ horas, su velocidad media es:

$$v_{\text{prom}} = \frac{\Delta x}{\Delta t} = \frac{100 \text{ km}}{2 \text{ h}} = 50 \text{ km/h}\,.$$

Sin embargo, este valor no nos dice a qué velocidad avanzaba el vehículo en un instante exacto t_0. En realidad, el velocímetro marca distintos valores durante el trayecto.

Para medir el cambio instantáneo, reducimos el intervalo de tiempo Δt haciéndolo cada vez más pequeño. La tasa de cambio en un instante exacto corresponde al límite:

$$v(t) = \lim_{\Delta t \to 0} \frac{x(t+\Delta t) - x(t)}{\Delta t} = \frac{dx}{dt}\,.$$

Geométricamente, la derivada es la pendiente de la recta tangente a la curva en un punto específico.

Suponga que se dispone de datos discretos (mediciones de tiempo t y posición x). Usando la rutina `np.gradient`, cuyo manual puede cosultar aquí podemos obtener la tasa de cambio instantánea en cada punto, veamos

derivada_MRUA.py

```
import numpy as np

def x(t, x0=0., v0 = 0., a=2.5):
    'MRUA'
    return x0 + v0*t + 1/2. *a*t**2

```

```
t = np.linspace(0,100,10)
x0 = float(input("Ingrese la posición inicial en m\n"))
v0 = float(input("Ingrese la velocidad inicial en m/s\n"))
a  = float(input("Ingrese la aceleración inicialen m0/s^2\n"))

r = x(t,x0,v0,a)

# velocidad v(t) = dx/dt
v_num = np.gradient(r, t)
# aceleración a(t) = dv/dt
a_num = np.gradient(v_num, t)


v_num2 = np.gradient(r, t, edge_order=2)
a_num2 = np.gradient(v_num2, t, edge_order=2)

# velocidad analítica v(t) = v0 + a * t
v_exacta = v0 + a*t
# aceleración analítica a(t) = a
a_exacta = a

print("\n" + "="*50)
print("Velocidad exacta:    \n", np.round(v_exacta, 2))
print("Velocidad numérica:  \n", np.round(v_num, 2))
print("-" * 50)
print("Aceleración exacta:  \n", a_exacta)
print("Aceleración numérica:\n", np.round(a_num, 2))
print("="*50)
print("Velocidad numérica edge_order2:  \n", np.round(v_num2, 2))
print("-" * 50)
print("Aceleración numérica edge_order2:\n", np.round(a_num2, 2))
print("="*50)
```

Cuando se trabaja con funciones en papel, la derivada se define mediante un límite infinitesimal. Sin embargo, numéricamente se trabaja con **muestras discretas** (mediciones tomadas a intervalos de tiempo Δt o espacio Δx). La función `np.gradient()` de NumPy nos permite calcular la tasa de cambio instantánea (derivada numérica) a lo largo de un arreglo de datos utilizando el método de **diferencias finitas** (este lo estudiaremos en detalle en la continuación de este curso). El algoritmo calcula la derivada tomando el cuenta los vecinos hacia adelante y haci atrás. En los extremos, no se tienen vecinos y pr eso los valores numéricos no coinciden con los análiticos. Note que con el argumento `edge_order =2` corregimos el error.

Ejemplo: la derivada de $\sin(x)$

derivada_sinx.py

```
import numpy as np
import matplotlib.pyplot as plt

```

```
N =  50 # número de úntos
x = np.linspace(0,2*np.pi,N)

y = np.sin(x)

dy = np.cos(x) # derivada exacta de sin(x)

# derivadas numéricas
dy1 =  np.gradient(y, x, edge_order=1)
dy2 =  np.gradient(y, x, edge_order=2)

#errores
error1 = np.abs(dy - dy1)
error2 = np.abs(dy - dy2)

#plots
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 10), sharex=True)

# Subplot 1: Función y derivadas
ax1.plot(x, y, "k--", alpha=0.5, lw = 2,label=r"$f(x) = \sin(x)$")
ax1.plot(x, dy, "k-", lw=2, label=r"Analítica: $\cos(x)$")
ax1.plot(x, dy1, "ro", markersize=5, label=r"Numérica (edge_order=1)")
ax1.plot(x, dy2, "b.", markersize=4, label=r"Numérica (edge_order=2)")
ax1.grid(True, ls=":", alpha=0.6)
ax1.legend(loc="lower left")

# Subplot 2: Error absoluto
ax2.semilogy(x, error1, "r-o", markersize=4, label=r"Error $|f'_{exacta} - f
    '_{num}|$ (edge=1)")
ax2.semilogy(x, error2, "b-s", markersize=4, label=r"Error $|f'_{exacta} - f
    '_{num}|$ (edge=2)")
ax2.set_xlabel(r"$x$ (rad)", fontsize=11)
ax2.set_ylabel("Error absoluto", fontsize=11)
ax2.grid(True, which="both", ls=":", alpha=0.6)
ax2.legend(loc="lower center")

plt.tight_layout()
#plt.savefig('../Clases/Herramientas_Python/derivadasinx.pdf', bbox_inches='
    tight')
plt.show()
```

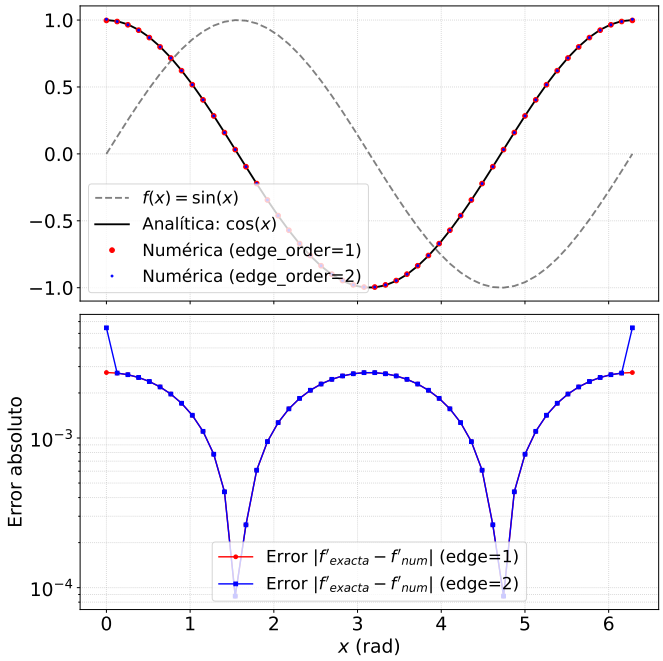


Figura 1: Derivada numérica de $f(x) = \sin(x)$.

Resumen del uso de `numpy.gradient`

♦ Acceda a la documentación oficial `scipy.optimize.root_scalar` dando clic aquí.

```
numpy.gradient(f, *varargs, axis=None, edge_order=1)
```

---

1. `f` (Obligatorio)

- El arreglo de datos (*1D, 2D o N-dimensional*) al que se le quiere calcular la derivada.

2. `*varargs` (Opcional)

- La variable independiente respecto a la que se deriva.
- **Formas de especificarlo:**
  - **Sin argumento:** Asume un espaciado unitario ($\Delta x = 1$).
  - **Un número escalar (`dx = 0.1`):** Indica que la distancia constante entre cada punto es 0.1.
  - **Un vector (`t`):** Pasa el arreglo de coordenadas exactas. Es la opción ideal si los puntos no están espaciados de forma uniforme.

3. `axis` (Opcional, por defecto `None`)

- Dirección o eje a lo largo del cual se deriva cuando se trabaja con matrices o arreglos multi-dimensionales.

- **Comportamiento:**
  - **En vectores 1D:** Se ignora o se deja por defecto.
  - **En matrices 2D:** `axis=None` calcula el gradiente en filas y columnas. `axis=0` deriva solo verticalmente; `axis=1` deriva solo horizontalmente.

4. `edge_order` (Opcional, por defecto 1)

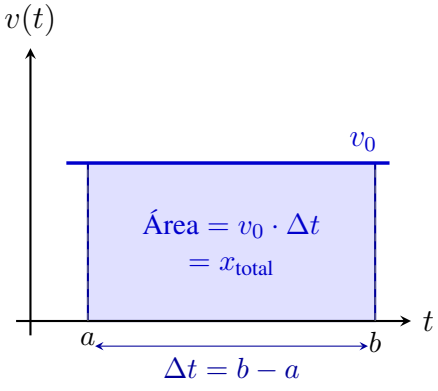
- Es el nivel de precisión empleado para evaluar la derivada **únicamente en los bordes** (primer y último punto).
- **Opciones:**
  - **`1`:** Usa 2 puntos para trazar una recta en el borde. Eficiente, pero introduce error en los extremos.
  - **`2`:** Usa 3 puntos para ajustar una parábola en el borde. Elimina el error en los extremos.

¿Qué devuelve la rutina?

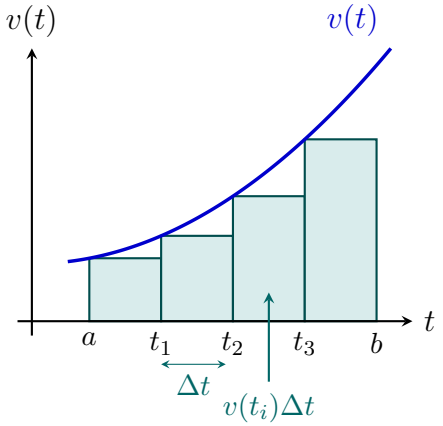
- Un nuevo `ndarray` del **mismo tamaño y forma** que `f` con los valores numéricos de la derivada en cada punto.

Integración

Consideremos ahora el problema inverso: conocemos la velocidad $v(t)$ en cada instante y queremos saber qué distancia total x se recorrió. Si la velocidad fuese constante, bastaría una multiplicación simple $x = v \cdot \Delta t$, que geométricamente equivale al área de un rectángulo, (figura 2a). Pero si la velocidad cambia con el tiempo (figura 2b), la multiplicación directa deja de funcionar.



(a) Velocidad constante: multiplicación directa.



(b) Velocidad variable: suma de rectángulos.

Figura 2: Convepto geométrico de integral.

Para calcular el total acumulado si la velocidad es variable:

1. Dividimos el intervalo continuo en subintervalos muy pequeños de ancho Δt.
2. Suponemos que en cada subintervalo la velocidad $v(t_i)$ es aproximadamente constante.
3. Sumamos el área de los delgados rectángulos resultantes: $x \approx \sum v(t_i)\Delta t$.

Al hacer los intervalos infinitesimalmente delgados ($\Delta t \to 0$), la suma discreta se transforma en una integral definida:

$$x_{\text{total}} = \int_a^b v(t)\, dt$$

Geométricamente, la integral representa el área bajo la curva $v(t)$ entre los límites $t = a$ y $t = b$. Podríamos decir, sin entrar en sutilezas matemáticas, que el cálculo se sintetiza en la relación fundamental entre ambas operaciones: ”la integral deshace la derivada y viceversa”.

$$\int_a^b \frac{dx(t)}{dt}\, dt = x(b) - x(a)$$

Ejemplo: Integración de $f(x) = x$

Esta integral es trivial y puede hacerse a ”mano”, note que si derivamos la funciópn $F(x) = \frac{1}{2}x^2 \implies F'(x) = x = f(x)$. Por el teorema fundamental del cálculo:

$$\int_a^b f(x)\, dx = \int_a^b F'(x)\, dx = F(x)|_a^b = \left.\frac{x^2}{2}\right|_a^b = \frac{b^2}{2} - \frac{a^2}{2}$$

Para $x \in [0, 2]$:

$$I = \int_0^2 x\, dx = \left.\frac{x^2}{2}\right|_0^2 = \frac{4}{2} - 0 = 2$$

```
In [1]: import scipy as sp

In [2]: import numpy as np

In [3]: f = lambda x: x

In [4]: a = 0.; b=2.

In [5]: integral,error = sp.integrate.quad(f, a, b)

In [6]: integral
Out[6]: 2.0

In [7]: error
Out[7]: 2.220446049250313e-14
```

Ejemplo: Integración de $f(x) = x^4 - 2x + 1$

En este caso, también se puede hacer a mano

$$\int_0^2 (x^4 - 2x + 1)dx = \frac{x^5}{5} - x^2 + x\Big|_0^2 = 4.4$$

```
In [1]: import scipy as sp

In [2]: f = lambda x: x**4 - 2*x + 1

In [3]: a = 0

In [4]: b = 2

In [5]: integral,error = sp.integrate.quad(f, a, b)

In [6]: integral
Out[6]: 4.3999999999999995

In [7]: error
Out[7]: 5.003996496050337e-14
```

Ejemplo: integral de $f(x) = e^{-x^2}$

Consideremos la función $f(x) = e^{-x^2}$ en el intervalo $[0, 1]$. Esta integral no posee primitiva expresable en términos de funciones elementales y define la función de error:

$$\int_0^1 e^{-x^2}\, dx = \frac{\sqrt{\pi}}{2}\,\text{erf}(1) \approx 0.746824132812427$$

En este tipo de problemas, el cálculo numérico no es solo una alternativa conveniente, sino una necesidad analítica.

```
In [2]: import scipy as sp

In [3]: f = lambda x: np.exp(-x**2)

In [4]: integral, error = sp.integrate.quad(f, 0.0, 1.0)

In [5]: integral
Out[5]: 0.746824132812427

In [6]: error
Out[6]: 8.291413475940725e-15
```

Ejemplo: integral de $f(x) = e^{-x^2}$

`scipy.integrate.quad` nos permite evaluar integrales impropias de primera especie, es decir, aquellas donde uno o ambos límites de integración se extienden al infinito:

$$\int_a^{\infty} f(x)\,dx, \qquad \int_{-\infty}^{b} f(x)\,dx, \qquad \int_{-\infty}^{\infty} f(x)\,dx$$

Bajo el capó, la subrutina `QAGI` de QUADPACK no discretiza un intervalo infinito arbitrario, sino que aplica transformaciones de variable racionales (por ejemplo, $x = \frac{1-t}{t}$ para mapear $[0, \infty)$ en $[0, 1]$) y evalúa la función mediante cuadraturas adaptativas de Gauss-Kronrod, eliminando problemas de divergencia por muestreo. En Python, indicamos los límites infinitos mediante la constante `np.inf` o `-np.inf`.

Por ejemplo, consideremos la integral

$$I = \int_{-\infty}^{\infty} e^{-x^2}\,dx = \sqrt{\pi} \approx 1.772453850905516$$

```
In [9]: import scipy as sp

In [10]: import numpy as np

In [11]: f = lambda x: np.exp(-x**2)

In [12]: integral,error = sp.integrate.quad(f, -np.inf, np.inf)

In [13]: integral
Out[13]: 1.7724538509055159

In [14]: error
Out[14]: 1.4202636780944923e-08
```

Ejemplo: integral de datos

Suponga que en un archivo `velocidades.txt` tiene guardado datos experimentales. Estos representan el tiempo y la velocidad. Recordando que la velocidad y la aceleración están dadas por $v = \dfrac{dx}{dt} \implies x = \int v dt$ $a = \dfrac{dv}{dt}$, podemos integrar y derivar los datos para obtener estas variables físicas.

integral_datos.py

```python
import numpy as np
import scipy as sp
import matplotlib.pyplot as plt

# Cargar datos del archivo
datos = np.loadtxt("velocidades.txt")
t = datos[:, 0]  # Tiempo en s
v = datos[:, 1]  # Velocidad en m/s
```

```
9
10 # Integración: trayectoria acumulada
11 # x(t) = integral_0^t v(t') dt'
12 # initial=0.0 asegura x(t0) = 0 y mantiene la misma dimensión que t
13 x_acum = sp.integrate.cumulative_trapezoid(v, x=t, initial=0.0)
14
15 # Distancia total recorrida
16 dist_total = sp.integrate.simpson(v, x=t)
17 print(f"Distancia total recorrida (método de Simpson): {dist_total:.4f} m")
18
19 # Derivación: aceleración instantánea a(t) = dv/dt
20 a_num = np.gradient(v, t, edge_order=2)
21
22 # Plots
23 fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(8, 9), sharex=True)
24
25 # Panel 1: Posición acumulada
26 ax1.plot(t, x_acum, "tab:red", lw=2, marker="s", markersize=4, label="
      Posición $x(t) = \\int v dt$")
27 ax1.set_ylabel("x (m)", fontsize=11)
28 ax1.grid(True, ls=":", alpha=0.6)
29 ax1.legend(loc="upper left")
30
31 # Panel 2: velocidad
32 ax2.plot(t, v, "tab:blue", lw=2, marker="o", markersize=4, label="Velocidad
      experimental $v(t)$")
33 ax2.set_ylabel("v (m/s)", fontsize=11)
34 ax2.grid(True, ls=":", alpha=0.6)
35 ax2.legend(loc="lower left")
36
37 # Panel 3: Aceleración
38 ax3.plot(t, a_num, "tab:green", lw=2, marker="^", markersize=4, label="
      Aceleración $a(t) = \\frac{dv}{dt}$ ")
39 ax3.set_xlabel("Tiempo (s)", fontsize=11)
40 ax3.set_ylabel("a (m/s$^2$)", fontsize=11)
41 ax3.grid(True, ls=":", alpha=0.6)
42 ax3.legend(loc="upper center")
43
44 #plt.savefig('../lectures/03_herramientas_python/integral_datos.pdf',
      bbox_inches='tight')
45
46 plt.tight_layout()
47 plt.show()
```

scipy.integrate

♦ Acceda a la documentación oficial `scipy.integrate` dando clic aquí.

`scipy.integrate.quad(func, a, b, args=())` (Funciones continuas)

`scipy.integrate.trapezoid(y, x=None, dx=1.0)` (Datos discretos, total)
`scipy.integrate.simpson(y, x=None, dx=1.0)` (Datos discretos, orden superior)
`scipy.integrate.cumulative_trapezoid(y, x=None, dx=1.0, initial=None)` (Integral acumulada)

---

1. `scipy.integrate.quad` (Cuadratura Adaptativa de Gauss-Kronrod)

Se utiliza cuando se conoce la **expresión matemática analítica** $f(x)$ y se desea evaluar $\int_a^b f(x)\,dx$.

- **`func` (Obligatorio):** Función ejecutable en Python que acepta como primer argumento la variable independiente x y retorna un valor escalar $f(x)$.
- **`a, b` (Obligatorio):** Límites inferior y superior de integración. Pueden ser números reales o infinitos (`-np.inf`, `np.inf`).
- **`args` (Opcional, tupla):** Parámetros adicionales que requiera la función $f(x, p_1, p_2)$ fuera de la variable de integración. Ejemplo: `args=(k, m)`.
- **Lo que devuelve:** Una tupla de dos valores: `(resultado, error_estimado)`, donde el segundo valor es una cota superior estricta del error absoluto numérico.

2. `scipy.integrate.trapezoid` (Regla del trapecio)

Aproxima el área bajo la curva uniendo puntos adyacentes con **segmentos lineales**. Ideal para datos experimentales o de sensores con variaciones suaves o pocos puntos.

- **`y` (Obligatorio):** Arreglo con las muestras dependientes (f_i).
- **`x` (Opcional):** Arreglo con la variable independiente. Es **imprescindible** si el muestreo no es uniforme (por ejemplo, tiempos variables t_i).
- **`dx` (Opcional, por defecto `1.0`):** Paso de integración escalar constante si no se especifica el vector `x`.
- **Lo que devuelve:** Un número escalar con la estimación acumulada del área total.

3. `scipy.integrate.simpson` (Regla de Simpson)

Aproxima el área ajustando **parábolas** sobre tríos de puntos consecutivos. Ofrece mayor precisión que el trapecio para funciones continuas con curvatura significativa.

- **`y, x, dx`:** Mismo uso y firma de parámetros que en `trapezoid`.
- **Requisito práctico:** Funciona mejor cuando los datos provienen de un muestreo uniforme (`dx` constante) y se dispone de un número impar de puntos (subintervalos pares).
- **Lo que devuelve:** Un número escalar con la estimación numérica acumulada del área total.

4. `scipy.integrate.cumulative_trapezoid` (Integral Acumulada / Trayectoria)

Evalúa la integral acumulativa paso a paso como una función del límite superior: $F(t) = \int_{t_0}^{t} y(\tau)\, d\tau$. A diferencia de `trapezoid` (que devuelve un único valor escalar del área total), esta rutina entrega un arreglo con la evolución temporal o espacial de la integral.

- **`y` (Obligatorio, primer argumento):** Arreglo con la **variable dependiente** a integrar (datos experimentales, velocidades $v(t)$, aceleraciones $a(t)$, tasas de producción, etc.).
- **`x` (Opcional, segundo argumento):** Arreglo con la **variable independiente** (eje temporal t o posición r). Es indispensable si el muestreo experimental no es regular (Δt variable).
- **`dx` (Opcional, por defecto `1.0`):** Espaciado constante entre muestras si no se provee el arreglo `x`.
- **`initial` (Crítico para dimensiones):** Dado que N mediciones definen únicamente $N-1$ intervalos de integración:
  - **`initial=None` (por defecto):** Devuelve un arreglo de longitud $N-1$, omitiendo el instante inicial t_0. Si se intenta graficar directamente contra `x`, Python arrojará un error de dimensiones (`ValueError`).
  - **`initial=0.0`:** Fija la condición $F(t_0) = \int_{t_0}^{t_0} y(\tau)\, d\tau = 0.0$. La salida conserva exactamente la longitud N, permitiendo graficar de inmediato pares $(t_i, F(t_i))$.
- **Lo que devuelve:** Un arreglo de NumPy con la serie integrada (por ejemplo, la trayectoria de posición $x(t)$ a partir de mediciones discretas de $v(t)$).

Criterio de selección

- **¿Tienes la fórmula analítica $f(x)$?** $\longrightarrow$ Usa `scipy.integrate.quad`.
- **¿Tienes datos discretos y buscas un valor total (escalar)?** $\longrightarrow$ Usa `trapezoid` o `simpson`.
- **¿Tienes datos discretos y quieres graficar la curva integrada en el tiempo?** $\longrightarrow$ Usa `cumulative_trapezoid(..., initial=0)`.

Problemas:

1. Considere la integral definida por la función:

$$E(x) = \int_0^x e^{-t^2}\, dt \tag{1}$$

Esta función no posee una antiderivada expresable en términos de funciones elementales, por lo que su evaluación requiere necesariamente de métodos de integración numérica.

Implemente un script en Python utilizando el ecosistema científico (`numpy`, `scipy.integrate` y `matplotlib.pyplot`) para resolver los siguientes numerales:

a) Cálculo numérico puntual: Escriba un programa que evalúe $E(x)$ para valores de x en el intervalo $[0, 3]$ con un paso de $\Delta x = 0.1$. Utilice la función `scipy.integrate.quad` para calcular la integral para cada valor de x. Imprima en consola una tabla con las columnas: x, $E(x)$ y la cota estimada de error arrojada por la rutina.

b) Visualización y asíntota teórica: Grafique $E(x)$ en función de x empleando marcadores en cada punto evaluado. Incluya en la misma gráfica una línea horizontal discontinua que represente el valor asintótico teórico cuando $x \to \infty$:

$$\lim_{x\to\infty} E(x) = \int_0^\infty e^{-t^2}\, dt = \frac{\sqrt{\pi}}{2}$$

Personalice la figura con etiquetas en los ejes, cuadrícula, título y leyenda.

c) Comparación con datos discretizados (Opcional): Para cada valor de x del literal (a), genere una partición uniforme de $N = 50$ subintervalos en $[0, x]$ y evalúe la integral utilizando la regla de Simpson mediante la rutina `scipy.integrate.simpson`. Calcule la diferencia absoluta entre el resultado de `simpson` y el obtenido con `quad`.

2. Considere la integral definida:

$$I = \int_0^1 \sin^2\left(\sqrt{100x}\right)\, dx \tag{2}$$

El argumento contiene una raíz escalada que provoca que la función presente múltiples oscilaciones en el intervalo $[0, 1]$, requiriendo un muestreo lo suficientemente fino para no subestimar las crestas y valles.

Escriba un programa en Python utilizando `scipy.integrate` y `numpy` para abordar los siguientes puntos:

a) Cálculo de alta precisión con cuadratura adaptativa: Calcule el valor de referencia de la integral utilizando la rutina `scipy.integrate.quad`. Configure una tolerancia relativa de $\varepsilon = 10^{-6}$ mediante el parámetro `epsrel=1e-6`. Imprima el resultado junto con la estimación del error.

b) Estudio de convergencia con datos discretos (`trapezoid` y `simpson`): Discretice el intervalo $[0, 1]$ duplicando sucesivamente el número de subdivisiones ($N = 2, 4, 8, 16, \ldots, 2048$). Para cada N:

- Evalúe la integral sobre la malla con `scipy.integrate.trapezoid`.
- Evalúe la integral sobre la malla con `scipy.integrate.simpson`.
- Calcule el error absoluto de cada método comparándolo con el valor obtenido en el literal (a).

Determine a partir de qué valor de N cada método alcanza la precisión deseada de $\varepsilon \leq 10^{-6}$.

c) Gráfica log-log de error vs. N: Genere una gráfica en escala doblemente logarítmica (`plt.loglog`) que muestre el error absoluto de `trapezoid` y de `simpson` en función de N. Compruebe visualmente si las pendientes corresponden al orden teórico esperado ($\mathcal{O}(h^2)$ para trapecio y $\mathcal{O}(h^4)$ para Simpson).

3. Considere una placa metálica cuadrada uniforme de lado $L = 10$ m, espesor despreciable y masa total $M = 10^4$ kg .

   a) Deducción analítica: Muestre que la componente perpendicular (z) de la fuerza gravitacional ejercida por la placa sobre una masa puntual de prueba $m = 1$ kg, ubicada a una distancia z sobre el centro geométrico de la placa, está dada por:

   $$F_z(z) = G\sigma z \int_{-L/2}^{L/2} \int_{-L/2}^{L/2} \frac{dx\, dy}{(x^2 + y^2 + z^2)^{3/2}} \tag{3}$$

   donde $G = 6.674 \times 10^{-11}$ m^3 kg^{-1} s^{-2} y $\sigma = M/L^2$ es la densidad superficial de masa.

   b) Evaluación numérica con `scipy.integrate.dblquad`: Escriba un programa en Python que calcule $F_z(z)$ para valores de z en el intervalo $[0, 10]$ m con un paso de $\Delta z = 0.1$ m. Utilice la función `scipy.integrate.dblquad` para evaluar la integral bidimensional en cada punto. Grafique F_z en función de z[cite: 4].

   c) Análisis físico y artefactos numéricos ($z \to 0$):

   - Analíticamente, para una placa infinita, el campo gravitacional en la superficie es constante: $F_z(0^+) = 2\pi G\sigma m$. Verifique si para distancias muy cortas ($z \ll L$) su curva se aproxima a este valor constante.
   - ¿Qué ocurre exactamente en $z = 0$? Explique la diferencia entre la discontinuidad física del problema (el campo cambia de signo al cruzar la placa) y la singularidad matemática en el integrando cuando $x = y = z = 0$.

